Engineering Certified JSON Derivers for Rocq with ELPI

Will Thomas and Perry Alexander

University of Kansas, Lawrence, USA
{30wthomas, palexand}@ku.edu

Abstract. Verified tools increasingly leave the proof assistant as extracted code that exchanges configuration, traces, counterexamples, and test data with ordinary infrastructure. JSON is a common format at that boundary, but the encoders and decoders used there are often hand-written and outside the checked development. We present `rocq-json`, an ELPI-based deriver that generates `Jsonifiable` instances for a practical fragment of Rocq inductive and record types. A generated instance contains an encoder, a decoder, and a Rocq-checked proof that decoding the encoded value recovers the original. The tool is distributed as an opam package, integrates with the `coq-elpi` derive framework, and includes an artifact that measures standard library applicability and extracted-code performance.

1 Introduction

Automated verification is ultimately useful when verified components run inside larger systems. A Rocq development may be extracted to OCaml, linked into a test harness, used as a runtime monitor, embedded in a model-checking workflow, or deployed as part of a verified tool chain. As soon as such a component communicates with the outside world, the proof assistant's internal data representation crosses an interface boundary.

JSON is a common boundary format for the kinds of data verified tools exchange: structured inputs, counterexample witnesses, execution traces, generated tests, runtime-monitor logs, and tool-pipeline artifacts. Extraction preserves the behavior of Rocq terms [8], but it does not certify the hand-written serializer that maps those terms to a deployment format. A bug at this layer can invalidate the engineering story of an otherwise verified pipeline.

Consider two extracted Rocq components: a verified analyzer that emits a counterexample witness and a verified replay checker that validates it. If the wire type is a trace datatype containing event records, branch choices, and final states, then the JSON representation is part of the proof boundary. An encoder that drops a constructor or a decoder that reads fields in a different order can make the replay checker validate a value different from the analyzer's internal witness. The individual proofs still hold, but the composition of the tools is mediated by unchecked boundary code.

`rocq-json` is aimed at this common tool-integration layer rather than at JSON as a data format in isolation. Verification tools often have highly structured internal datatypes and ordinary external workflows: they read configuration files, hand results to scripts, archive traces for later inspection, or compose with independently developed checkers. Those workflows are attractive precisely because JSON is mundane and well-supported outside the proof assistant. The risk is that the mundane layer becomes the place where a verified value is silently reshaped by unverified code. A deriver that produces both executable serializers and a checked theorem makes this boundary explicit and repeatable for every datatype it supports.

`rocq-json` addresses this gap by deriving serializers together with the round-trip theorem needed by this producer-to-consumer use case. The public contract is the following typeclass:

```
Class Jsonifiable (A : Type) := {
  to_JSON    : A → JSON;
  from_JSON  : JSON → Result A string;
  canonical_jsonification : forall (a : A), from_JSON (to_JSON a) = res a
}.
```

The deriver generates all three components for many ordinary inductive and record types and exposes them through the `coq-elpi` derive framework [7]. The contribution is a practical verification engineering tool: it extends the checked trust perimeter of extracted Rocq components to the JSON boundary while keeping the generated code suitable for extraction.

The substantially new combination is: ELPI inspection of Rocq declarations, generated encoders and decoders, typeclass parameters for nested serializable fields, automatic proof construction for the round-trip theorem, and a widely applicable tool for easy and correct automated derivation. The tool is available as the `rocq-json` opam package; the development repository and artifact are at `https://github.com/ku-sldg/rocq-json`.

2 Tool Interface and Design

The user-facing interface is deliberately small. Given an enumeration, algebraic datatype, parameterized container, or record, the user invokes the deriver and receives a registered typeclass instance. Nullary constructors are encoded as JSON strings. Constructors with arguments are encoded as singleton JSON objects keyed by constructor name, with named fields in the nested object. Records are encoded as JSON objects keyed by projection names. Appendix B gives an install-and-use guide, and Appendix C summarizes the supported fragment and encoding conventions.

```
Inductive UserRole : Type :=
| Admin | Guest | Moderator (level : nat)
| StandardUser (id : nat) (username : string).
Elpi derive.jsonifiable UserRole.
```

```
Example role_roundtrip (r : UserRole) :
  from_JSON (to_JSON r) = res r := canonical_jsonification r.
Example moderator_json :
  to_JSON (Moderator 2) = JSON_Object [
    ("Moderator", JSON_Object [("level", JSON_Nat 2)])] := eq_refl.
```

For parameterized types, the generated instance takes serializers for type parameters. For recursive types, the generated encoder and decoder use structurally recursive definitions. The implementation also supports a conservative collection of nested recursive occurrences through standard data containers such as lists, options, and products. More general nested or mutually recursive types are reported as unsupported rather than handled partially.[1]

The generated theorem is intentionally one-sided. It says that JSON emitted by the encoder can be decoded back to the original Rocq value. This is the direction needed when a verified component produces JSON for another tool or service. The converse property, that every accepted JSON value is canonical, would require a representation-specific grammar and rejection theorem for each type. That is useful future work, but it is a different contract from the boundary-preservation guarantee provided here.

The interface is intentionally conservative in its failure behavior. When the deriver recognizes a shape outside the current fragment, such as a dependent constructor field or an indexed family, it reports a Rocq error rather than registering a partial instance. A failed derivation leaves the development unchanged, while a successful derivation gives ordinary typeclass operations and the checked round-trip proof. The artifact's batch probe uses the same command, so its applicability numbers reflect the interface users actually invoke.

3 Implementation

The deriver is implemented in `coq-elpi`, an embedded λProlog language for Rocq metaprogramming. It reads the inductive or record view of the target, classifies parameters and constructor arguments, and builds skeletons for the encoder, decoder, proof, and final typeclass instance. Rocq elaborates and checks the generated terms before they become constants, so the trusted result is not the ELPI program itself but the generated Rocq definitions and the accepted `canonical_jsonification` theorem.

Two engineering choices are central to tool quality. First, field serializers are kept abstract through typeclass arguments where possible. Generated instances therefore compose with existing hand-written or derived instances for primitive and container types; for example, `nat` is serialized as a JSON number rather than by exposing its unary inductive representation. Second, decoder generation

[1] Generating useful `to/from_JSON` functions is easier than proving them correct without strong structural recursion support. Better support for nested inductives in Rocq is active work [3].

is extraction-conscious. Large enumerations are decoded with a string decision tree that extracts to direct pattern matching over characters, avoiding a long chain of string equality tests.

Proof generation is mixed. General recursive and record cases use Rocq proof automation specialized to the generated encoders and decoders. Pure enumerations use direct generated proof terms, avoiding proof-search overhead on a common stress case. This split is not theoretically deep, but it matters for stable tool latency and scaling.

The design separates two trust questions. The ELPI code finds declaration structure and proposes Rocq terms; Rocq accepts or rejects those terms. A bug in the deriver may make a command fail, generate inefficient code, or choose an inconvenient representation, but it does not by itself prove a false round-trip theorem. Metaprogramming improves coverage and ergonomics, while the kernel checks the artifacts that enter the development.

4 Evaluation

We evaluate two practical questions: whether the deriver applies to a meaningful fragment of existing Rocq declarations, and whether generated code has the same general extracted-performance profile as direct handwritten Rocq serializers. The artifact rebuilds the project, scans installed Corelib/Stdlib sources, classifies declarations by whether the `Jsonifiable` contract is a well-defined request, compiles all successful target-fragment derivations together, runs extracted OCaml benchmarks against handwritten Rocq baselines, and regenerates the LaTeX macros used in this paper. Appendix A gives the full methodology, raw diagnostic categories, and benchmark code.

Table 1. Interpreted Corelib/Stdlib applicability.

Metric	Count	Metric	Count
Discovered source declarations	342	Successful target derivations	81
Not closed deriver inputs	31	Current limitations	72
Deliberate semantic non-targets	158	Timeouts	0
Well-defined target inputs	153		

Table 1 separates applicability from diagnostics. The raw artifact log contains every scanner candidate and every rejected probe, but the correct denominator for tool applicability is not all closed declarations in Corelib/Stdlib. Propositions, arbitrary functions, sort-valued fields, and coinductive data are deliberate semantic non-targets for generic runtime JSON serialization, while declarations inside functors or signatures are not closed global references to which the deriver can be applied directly. After excluding those cases, the reported target fragment contains 153 inputs. The tool derives 81 of them, 53% coverage, with the remaining gap concentrated in dependent fields, indexed families, and proof-bearing computational wrappers.

The successful derivations in Table 1 include 50 inductives, 17 variants, and 14 records or structures, split between Corelib (39) and Stdlib (42).

Successful derivations are fast in the reported run. The combined successful benchmark compiled in 5.26 seconds; the mean Rocq-reported derivation transaction was 0.054 seconds, the median was 0.033 seconds, and the maximum was 0.500 seconds. The slowest success was `byte`, a 256-constructor Corelib enumeration and the broad benchmark's version of the large-enum stress case used while engineering the string decision tree.

Table 2. Extracted round-trip benchmarks.

Benchmark	Generated mean (ms)	Handwritten mean (ms)	Change (%)
enum256	43.917	43.052	+2.0%
point2d	1.066	1.170	-8.9%
userrole	1.790	2.648	-32.4%
serverconfig	0.635	1.525	-58.4%
instruction	1.733	2.666	-35.0%
bintree	3.321	5.301	-37.3%

Table 2 compares generated definitions with handwritten Rocq definitions after both are extracted to OCaml. Negative changes mean the generated code is faster than the handwritten baseline. The comparison should not be read as a claim of superiority over all possible serializers or OCaml-specific deriving systems. Its purpose is narrower and tool-focused: generated certified serializers avoid obvious extraction-time performance regressions on representative shapes.

The important evaluation point is not that every Rocq declaration should have a generic JSON encoding. Many declarations are logical, type-level, functional, or infinite in ways that do not correspond to ordinary runtime data. The useful evidence is instead that the tool can be run systematically over standard libraries, unsupported inputs are visible and classified, successful derivations remain fast, and extracted code does not pay an obvious price for carrying a checked source-level contract.

5 Related Work and Limitations

Rocq has several metaprogramming routes for deriving boilerplate. MetaCoq provides quoted syntax and verified metaprogramming [6]; `coq-elpi` provides an embedded logic-programming language integrated with Rocq elaboration and an existing derive framework. `rocq-json` uses `coq-elpi` because the task is structural inspection plus term construction, with Rocq checking the generated executable definitions and theorem. Compared with typical derivers for equality, ordering, showing, or decidability, this deriver generates two programs and a theorem relating them.

This work is distinct from Rocq/JSON infrastructure such as SerAPI [1]. SerAPI serializes Rocq's own syntax and protocol data for external clients, while `rocq-json` derives certified JSON instances for user-defined Rocq data. Lean's

JSON support and typeclass-based derivation facilities [5, 4] and OCaml libraries such as `ppx_yojson_conv` [2] support practical serialization workflows, but they do not normally produce a source-level theorem enforcing semantics on the decoder and encoder.

The current deriver targets computational data in `Type` or `Set`. It deliberately excludes shapes where generic serialization is not a well-defined request: propositions are erased by extraction, arbitrary functions have no canonical finite data representation, sort-valued fields are types rather than values, and coinductive values need a bounded or lazy contract. The most important future work is support for dependent and indexed data, likely through fixed-index instances, dependent-pair decoding, or user-supplied proof reconstruction where extraction erases evidence.

6 Conclusion

`rocq-json` demonstrates that JSON boundary code for extracted verified components can be treated as part of the formal artifact. Its ELPI deriver generates executable encoders and decoders together with Rocq-checked round-trip proofs, applies across a meaningful standard-library fragment, and extracts to competitive code on representative examples. This moves an ordinary integration layer back inside the checked assurance boundary.

References

1. Gallego Arias, E.J.: SerAPI: Machine-Friendly, Data-Centric Serialization for Coq. Tech. rep., MINES ParisTech (Oct 2016), `https://hal-mines-paristech.archives-ouvertes.fr/hal-01384408`
2. Jane Street: ppx_yojson_conv. `https://github.com/janestreet/ppx_yojson_conv`, accessed 2026-05-05
3. Lamiaux, T., Forster, Y., Sozeau, M., Tabareau, N.: Nested Inductive Types (2025), `https://hal.science/hal-05366368`, working paper or preprint
4. Lean developers: Lean 4 JSON support. `https://lean-lang.org/doc/api/Lean/Data/Json/Basic.html`, accessed 2026-05-05
5. de Moura, L., Kong, S., Avigad, J., van Doorn, F., von Raumer, J.: The lean theorem prover (system description). In: Felty, A.P., Middeldorp, A. (eds.) Automated Deduction - CADE-25. pp. 378–388. Springer International Publishing, Cham (2015)
6. Sozeau, M., Anand, A., Boulier, S., Cohen, C., Forster, Y., Kunze, F., Malecha, G., Tabareau, N., Winterhalter, T.: The MetaCoq Project. Journal of Automated Reasoning (Feb 2020). https://doi.org/10.1007/s10817-019-09540-0, `https://inria.hal.science/hal-02167423`
7. Tassi, E.: Elpi: rule-based meta-language for Rocq. In: CoqPL 2025 - The Eleventh International Workshop on Coq for Programming Languages. Denver, United States (Jan 2025), `https://inria.hal.science/hal-04990628`
8. The Rocq Development Team: The Rocq Prover. Zenodo, Version 9.0.0, `10.5281/zenodo.11551307` (Apr 2025). https://doi.org/10.5281/zenodo.15149629

A Benchmarking Details

This appendix records the benchmark methodology and supporting code behind Section 4. The intent is reproducibility rather than a new claim beyond the main paper: the artifact reports both successful derivations and unsupported cases, and it compares generated serializers against handwritten Rocq baselines extracted through the same pipeline.

A.1 Applicability Benchmark

The artifact scans installed Corelib and Stdlib sources for inductive-like declarations, records declarations that are not closed deriver inputs, probes closed candidates in isolation, records diagnostics, and then compiles a single benchmark file containing all successful derivations. This workflow is deliberately conservative: semantic non-targets and unsupported target-fragment declarations remain visible as categorized diagnostics. Declarations that are visible in source but are not closed derivation targets, such as declarations inside functor bodies or module signatures, are recorded separately rather than discarded.

Table 3. Corelib/Stdlib raw applicability log and diagnostic categories.

Metric	Count
Discovered declarations	342
Recorded, not probed	31
Probed declarations	311
Successful probes	81
Non-success diagnostics	230
Probe timeouts	0
Timed successful derivations	81

Diagnostic category	Count
prop-target-not-supported	150
dependent-field-not-supported	48
indexed-family-not-supported	21
function-field-not-supported	4
proof-field-not-supported	3
sort-field-not-supported	3
coinductive-not-supported	1

The category names in Table 3 are a triage device, not a denominator for tool applicability. Some declarations are clear non-targets for serialization: propositions are erased by extraction, functions have no canonical JSON representation, sort-valued fields contain types rather than values, and coinductive values may be infinite. The genuine current limitations are concentrated around dependent data, where decoding must reconstruct values whose types depend on earlier fields or external indices. Table 4 gives one representative declaration for each category and the rationale used by the artifact.

The non-target group is not a design choice particular to `rocq-json`: there is no runtime proposition to serialize after extraction; arbitrary functions do not have a canonical finite data representation; sorts such as `Type` and `Set` are not ordinary values; and coinductive data needs a separate bounded or lazy contract. These account for 158 diagnostics that should be screened out before computing target-fragment coverage. The remaining 72 target-fragment diagnostics are dependent fields, indexed families, and proof-bearing computational wrappers. Those are real future work: the JSON decoder would have to recover enough

Table 4. Representative failure examples and classification rationale.

Category and example	Rationale
`prop-target-not-supported: or`	The target lives in `Prop`; propositions are erased at extraction and are not runtime data.
`dependent-field-not-supported: sig`	A later field depends on an earlier witness; decoding must reconstruct a value and its dependent type.
`indexed-family-not-supported: reflect`	The declaration is indexed by values; decoding requires recovering the index or fixing it in the instance.
`function-field-not-supported: implies`	Arbitrary functions have no canonical data representation in JSON or ordinary serializers.
`proof-field-not-supported: sumbool`	A computational wrapper carries proof evidence; decoding requires proof erasure or reconstruction.
`sort-field-not-supported: Tlist`	Constructor fields contain sorts such as `Type`; these are types, not JSON values.
`coinductive-not-supported: Stream`	Potentially infinite data needs an explicit depth-bounded or lazy serialization contract.

information to reconstruct a dependent value, the deriver would need a fixed-index or sigma-style encoding, or proof-bearing data would need erasure plus reconstruction lemmas.

The 31 recorded-but-not-probed declarations are reported in the artifact as well: 24 occur inside parameterized modules or functor bodies, such as the raw tree declaration in `FMapAVL`, and 7 occur inside module types or signatures, such as the tree declaration in `MSetGenTree.Ops`. These are genuine source declarations, but not closed global references to which the deriver can be applied directly *outside the functor*.

Excluding the 31 functor/signature declarations that are not valid deriver inputs and the 158 semantic non-targets, the tool successfully derives instances for 81 of 153 semantically eligible declarations, roughly 53% coverage on the well-defined target fragment.

A.2 Derivation Time

The combined successful benchmark records Rocq's `Time` output for each derived instance. In the reported run, the combined benchmark compiled successfully with wall-clock time 5.26 seconds; the sum of Rocq-reported derivation transactions was 4.34 seconds. The mean derivation time was 0.054 seconds, the median was 0.033 seconds, the 90th percentile was 0.101 seconds, and the maximum was 0.500 seconds.

The slowest success is `byte`, a 256-constructor Corelib enumeration. The remaining cost is dominated by Rocq-side derivation and proof construction. After extraction, the corresponding dispatch path is close to the handwritten Rocq baseline used in the extracted-code benchmark.

A.3 Extracted Code Performance

The extraction benchmark compares generated definitions against handwritten Rocq definitions after both have been extracted to OCaml. This keeps the comparison in the same source language and extraction pipeline. The benchmark

Table 5. Slowest successful derivations in the broad benchmark.

Declaration	Kind	Seconds
Corelib.Init.Byte.byte	Inductive	0.500
Stdlib.micromega.ZMicromega.ZArithProof	Inductive	0.190
Stdlib.rtauto.Rtauto.proof	Inductive	0.180
Stdlib.btauto.Reflect.formula	Inductive	0.178
Stdlib.micromega.RingMicromega.Psatz	Inductive	0.139
Corelib.Floats.FloatClass.float_class	Variant	0.136
Stdlib.setoid_ring.Field_theory.FExpr	Inductive	0.120
Stdlib.Sorting.Heap.Tree	Inductive	0.118

covers several shapes: a 256-constructor enumeration, a tuple-like constructor, a flat record, two mixed sums with nullary and field-bearing constructors, and a recursive binary tree. Each benchmark was run 10 times. The enum benchmark serializes and deserializes every constructor; the other benchmarks run many iterations over representative values. The handwritten baselines were written by the authors in Rocq and extracted through the same pipeline; they were not independently optimized.

A.4 Round-Trip Benchmark Rocq Code

The extracted-code benchmarks in Table 2 are defined in the extraction benchmark source. The generated cases use `Elpi derive.jsonifiable`; the primed instances below are the handwritten Rocq baselines extracted and timed by the same harness.

Benchmark Types

```
(* 1.3. Single Constructor, Multiple Arguments (Tuple-like) *)
Inductive Point2D : Type :=
| MkPoint2D (x : nat) (y : nat).
Elpi derive.jsonifiable Point2D.

(* 1.4. Mixed Arity Sum Type *)
Inductive UserRole : Type :=
| Admin
| Moderator (level : nat)
| StandardUser (id : nat) (username : string)
| Guest.
Elpi derive.jsonifiable UserRole.

(* 1.5. Flat Record with primitive types *)
Record ServerConfig : Type := {
  host : string;
  port : nat;
  use_ssl : bool
}.
Elpi derive.jsonifiable ServerConfig.
```

```
(* 1.9. Flat Instruction Set (Mix of no-args and primitive args) *)
Inductive Instruction : Type :=
| INop
| IPush (val : nat)
| IPop
```

```
| ILoad (reg_idx : nat)
| IStore (reg_idx : nat)
| IHalt.
Elpi derive.jsonifiable Instruction.
```

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).
Elpi derive.jsonifiable BinTree.
```

```
Inductive enum_256 :=
| e00 | e01 | e02 | e03 | e04 | e05 | e06 | e07
| e08 | e09 | e0A | e0B | e0C | e0D | e0E | e0F
| e10 | e11 | e12 | e13 | e14 | e15 | e16 | e17
| e18 | e19 | e1A | e1B | e1C | e1D | e1E | e1F
| e20 | e21 | e22 | e23 | e24 | e25 | e26 | e27
| e28 | e29 | e2A | e2B | e2C | e2D | e2E | e2F
| e30 | e31 | e32 | e33 | e34 | e35 | e36 | e37
| e38 | e39 | e3A | e3B | e3C | e3D | e3E | e3F
| e40 | e41 | e42 | e43 | e44 | e45 | e46 | e47
| e48 | e49 | e4A | e4B | e4C | e4D | e4E | e4F
| e50 | e51 | e52 | e53 | e54 | e55 | e56 | e57
| e58 | e59 | e5A | e5B | e5C | e5D | e5E | e5F
| e60 | e61 | e62 | e63 | e64 | e65 |  e66 | e67
|  e68 | e69 | e6A | e6B | e6C | e6D | e6E | e6F
|  e70 | e71 | e72 | e73 | e74 | e75 | e76 | e77
|  e78 | e79 | e7A | e7B   |  e7C   |  e7D   |  e7E   |  e7F
| e80 | e81 | e82 | e83 | e84 | e85 | e86 | e87
| e88 | e89 | e8A | e8B | e8C | e8D | e8E | e8F
| e90 | e91 | e92 | e93 | e94 | e95 | e96 | e97
| e98 | e99 | e9A | e9B | e9C | e9D | e9E | e9F
| eA0 | eA1 | eA2 | eA3 | eA4 | eA5 | eA6 | eA7
| eA8 | eA9 | eAA | eAB | eAC | eAD | eAE | eAF
| eB0 | eB1 | eB2 | eB3 | eB4 | eB5 | eB6 | eB7
| eB8 | eB9 | eBA | eBB | eBC | eBD | eBE | eBF
| eC0 | eC1 | eC2 | eC3 | eC4 | eC5 | eC6 | eC7
| eC8 | eC9 | eCA | eCB | eCC | eCD | eCE | eCF
| eD0 | eD1 | eD2 | eD3 | eD4 | eD5 | eD6 | eD7
| eD8 | eD9 | eDA | eDB | eDC | eDD | eDE | eDF
| eE0 | eE1 | eE2 | eE3 | eE4 | eE5 | eE6 | eE7
| eE8 | eE9 | eEA | eEB | eEC | eED | eEE | eEF
| eF0 | eF1 | eF2 | eF3 | eF4 | eF5 | eF6 | eF7
| eF8 | eF9 | eFA | eFB | eFC | eFD | eFE | eFF.
Time Elpi derive.jsonifiable enum_256.
```

Handwritten Rocq Baselines

```
Definition Point2D_Jsonifiable' '{JN : Jsonifiable nat} : Jsonifiable Point2D.
refine (Build_Jsonifiable _ (fun p ⇒
  match p with
  | MkPoint2D x y ⇒
      JSON_Object [("MkPoint2D", JSON_Object [("x", to_JSON x); ("y", to_JSON y)])]
  end) (fun js ⇒
  match js with
  | JSON_Object [("MkPoint2D", JSON_Object [("x", xv); ("y", yv)])] ⇒
      x ← from_JSON xv ;;
      y ← from_JSON yv ;;
      res (MkPoint2D x y)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn.
erewrite canonical_jsonification.
erewrite canonical_jsonification.
reflexivity.
Defined.

Definition UserRole_Jsonifiable' '{JN : Jsonifiable nat, JS : Jsonifiable string} :
     Jsonifiable UserRole.
refine (Build_Jsonifiable _ (fun r ⇒
  match r with
```

```
  | Admin ⇒ JSON_String "Admin"
  | Moderator level ⇒
      JSON_Object [("Moderator", JSON_Object [("level", to_JSON level)])]
  | StandardUser id username ⇒
      JSON_Object [("StandardUser", JSON_Object [("id", to_JSON id); ("username", to_JSON
     username)])]
  | Guest ⇒ JSON_String "Guest"
  end) (fun js ⇒
  match js with
  | JSON_String "Admin" ⇒ res Admin
  | JSON_String "Guest" ⇒ res Guest
  | JSON_Object [("Moderator", JSON_Object [("level", jlevel)])] ⇒
      level ← from_JSON jlevel ;;
      res (Moderator level)
  | JSON_Object [("StandardUser", JSON_Object [("id", jid); ("username", usernamev)])] ⇒
      id ← from_JSON jid ;;
      username ← from_JSON usernamev ;;
      res (StandardUser id username)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn;
repeat (erewrite canonical_jsonification);
try reflexivity.
Defined.

Definition ServerConfig_Jsonifiable'
    `{JS : Jsonifiable string, JN : Jsonifiable nat, JB : Jsonifiable bool}
    : Jsonifiable ServerConfig.
refine (Build_Jsonifiable _ (fun cfg ⇒
  match cfg with
  | {| host := h; port := p; use_ssl := ssl |} ⇒
      JSON_Object [("host", to_JSON h); ("port", to_JSON p); ("use_ssl", to_JSON ssl)]
  end) (fun js ⇒
  match js with
  | JSON_Object [("host", jhost); ("port", jport); ("use_ssl", jssl)] ⇒
      h ← from_JSON jhost ;;
      p ← from_JSON jport ;;
      ssl ← from_JSON jssl ;;
      res {| host := h; port := p; use_ssl := ssl |}
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn.
repeat (erewrite canonical_jsonification).
reflexivity.
Defined.

Definition Instruction_Jsonifiable' `{JN : Jsonifiable nat} : Jsonifiable Instruction.
refine (Build_Jsonifiable _ (fun i ⇒
  match i with
  | INop ⇒ JSON_String "INop"
  | IPush val ⇒ JSON_Object [("IPush", JSON_Object [("val", to_JSON val)])]
  | IPop ⇒ JSON_String "IPop"
  | ILoad reg_idx ⇒ JSON_Object [("ILoad", JSON_Object [("reg_idx", to_JSON reg_idx)])]
  | IStore reg_idx ⇒ JSON_Object [("IStore", JSON_Object [("reg_idx", to_JSON reg_idx)])]
  | IHalt ⇒ JSON_String "IHalt"
  end) (fun js ⇒
  match js with
  | JSON_String "INop" ⇒ res INop
  | JSON_String "IPop" ⇒ res IPop
  | JSON_String "IHalt" ⇒ res IHalt
  | JSON_Object [("IPush", JSON_Object [("val", jval)])] ⇒
      val ← from_JSON jval ;;
      res (IPush val)
  | JSON_Object [("ILoad", JSON_Object [("reg_idx", jreg_idx)])] ⇒
      reg_idx ← from_JSON jreg_idx ;;
      res (ILoad reg_idx)
  | JSON_Object [("IStore", JSON_Object [("reg_idx", jreg_idx)])] ⇒
      reg_idx ← from_JSON jreg_idx ;;
```

```
      res (IStore reg_idx)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end) _).
destruct a; cbn;
repeat (erewrite canonical_jsonification);
try reflexivity.
Defined.

Fixpoint bintree_to_JSON' {A : Type} `{Jsonifiable A} (t : BinTree A) : JSON :=
  match t with
  | BinLeaf _ ⇒ JSON_String "BinLeaf"
  | BinNode _ l value r ⇒
      JSON_Object [("BinNode", JSON_Object [
        ("left", bintree_to_JSON' l);
        ("value", to_JSON value);
        ("right", bintree_to_JSON' r)])]
  end.

Fixpoint bintree_from_JSON' {A : Type} `{Jsonifiable A} (js : JSON) : Result (BinTree A)
     string :=
  match js with
  | JSON_String "BinLeaf" ⇒ res (·BinLeaf A)
  | JSON_Object [("BinNode", JSON_Object [("left", jleft); ("value", jvalue); ("right",
     jright)])] ⇒
      left ← bintree_from_JSON' jleft ;;
      value ← from_JSON jvalue ;;
      right ← bintree_from_JSON' jright ;;
      res (·BinNode A left value right)
  | _ ⇒ err err_str_json_unrecognized_constructor
  end.

Definition BinTree_Jsonifiable' {A : Type} `{Jsonifiable A} : Jsonifiable (BinTree A).
refine (Build_Jsonifiable _ bintree_to_JSON' bintree_from_JSON' _).
induction a; cbn.
- reflexivity.
- rewrite IHa1.
  rewrite canonical_jsonification.
  rewrite IHa2.
  reflexivity.
Defined.
```

```
Definition enum_256_Jsonifiable' : Jsonifiable enum_256.
Time refine (Build_Jsonifiable _ (fun e ⇒ match e with
  | e00 ⇒ JSON_String "e00" | e01 ⇒ JSON_String "e01"
  | e02 ⇒ JSON_String "e02" | e03 ⇒ JSON_String "e03"
  | e04 ⇒ JSON_String "e04" | e05 ⇒ JSON_String "e05"
  | e06 ⇒ JSON_String "e06" | e07 ⇒ JSON_String "e07"
  | e08 ⇒ JSON_String "e08" | e09 ⇒ JSON_String "e09"
  | e0A ⇒ JSON_String "e0A" | e0B ⇒ JSON_String "e0B"
  | e0C ⇒ JSON_String "e0C" | e0D ⇒ JSON_String "e0D"
  | e0E ⇒ JSON_String "e0E" | e0F ⇒ JSON_String "e0F"
  | e10 ⇒ JSON_String "e10" | e11 ⇒ JSON_String "e11"
```

⋮

```
  | eFE ⇒ JSON_String "eFE" | eFF ⇒ JSON_String "eFF"
  end) (fun e ⇒
    match e with
  | JSON_String "e00" ⇒ res e00 | JSON_String "e01" ⇒ res e01
  | JSON_String "e02" ⇒ res e02 | JSON_String "e03" ⇒ res e03
  | JSON_String "e04" ⇒ res e04 | JSON_String "e05" ⇒ res e05
  | JSON_String "e06" ⇒ res e06 | JSON_String "e07" ⇒ res e07
  | JSON_String "e08" ⇒ res e08 | JSON_String "e09" ⇒ res e09
  | JSON_String "e0A" ⇒ res e0A | JSON_String "e0B" ⇒ res e0B
  | JSON_String "e0C" ⇒ res e0C | JSON_String "e0D" ⇒ res e0D
  | JSON_String "e0E" ⇒ res e0E | JSON_String "e0F" ⇒ res e0F
  | JSON_String "e10" ⇒ res e10 | JSON_String "e11" ⇒ res e11
```

⋮

```
  | JSON_String "eFE"  ⇒ res eFE | JSON_String "eFF" ⇒ res eFF
  | _ ⇒ err err_str_json_unrecognized_constructor
  end
  ) _); destruct a; reflexivity.
Qed.
```

Extraction Command

```
From Corelib Require Import Extraction.

Extraction "jsonifiable_suite.ml"
  enum_256_Jsonifiable enum_256_Jsonifiable'
  Point2D_Jsonifiable Point2D_Jsonifiable'
  UserRole_Jsonifiable UserRole_Jsonifiable'
  ServerConfig_Jsonifiable ServerConfig_Jsonifiable'
  Instruction_Jsonifiable Instruction_Jsonifiable'
  BinTree_Jsonifiable BinTree_Jsonifiable'.
```

B Quick Start Guide

Install the released package with opam:

```
opam repo add -a --set-default ku-sldg/opam-repo https://github.com/ku-
    sldg/opam-repo.git
opam repo add coq-released https://coq.inria.fr/opam/released
opam install rocq-json
```

Import the JSON library and deriver, define a datatype, and invoke the derive command:

```
From RocqJSON Require Import JSON JSON_Derive.

Inductive UserRole : Type :=
| Admin
| Moderator (level : nat)
| Guest.

Elpi derive.jsonifiable UserRole.

Example moderator_roundtrip :
  from_JSON (to_JSON (Moderator 2)) = res (Moderator 2) :=
  canonical_jsonification (Moderator 2).
```

For a parameterized type, derive the type constructor. The generated instance takes serializers for the parameters:

```
Inductive BinTree (A : Type) : Type :=
| BinLeaf
| BinNode (left : BinTree A) (value : A) (right : BinTree A).

Elpi derive.jsonifiable BinTree.

Check BinTree_Jsonifiable :
  forall A, Jsonifiable A → Jsonifiable (BinTree A).
```

If derivation is requested outside the supported contract, the command raises an ordinary Rocq error. Deliberate non-targets include targets in `Prop`, function-valued fields, sort-valued fields, and coinductive declarations. Current limitations include dependent constructor fields, indexed families, and proof-bearing computational wrappers. A typical dependent-field diagnostic looks like:

```
Error:
derive.jsonifiable: dependent constructor fields are not supported by
Jsonifiable. Field: ... type head: unknown type constructor
```

C Reference Manual

C.1 Public Contract

`Jsonifiable` packages an encoder, decoder, and one checked theorem:

```
Class Jsonifiable (A : Type) := {
  to_JSON    : A → JSON;
  from_JSON  : JSON → Result A string;
  canonical_jsonification : forall (a : A), from_JSON (to_JSON a) = res a
}.
```

The theorem states encoder-to-decoder preservation. It does not state that every JSON value accepted by the decoder is the unique canonical representation for the type.

C.2 Supported Declarations

The deriver targets computational declarations in `Type` or `Set`. It supports ordinary enumerations, algebraic datatypes, records, parameterized types whose parameters have `Jsonifiable` instances, and structural recursion over the target. The current implementation also supports a conservative set of nested recursive occurrences through standard containers such as lists, options, and products.

Declarations outside the contract fail without registering a partial instance. Deliberate semantic non-targets include targets in `Prop`, function-valued fields, sort-valued fields, and coinductive declarations. Current implementation limitations include dependent constructor fields, indexed families, proof-bearing computational wrappers, arbitrary nested recursion, and mutual recursion.

C.3 Encoding Conventions

Rocq shape	JSON shape
Nullary constructor	JSON string containing the constructor name.
Constructor with fields	Singleton JSON object keyed by constructor name.
Record	JSON object keyed by projection names.
Named constructor argument	Field key is the binder name.
Anonymous constructor argument	Field key is derived from the argument type head and disambiguated with numeric suffixes.
Type parameter	Instance argument requiring a `Jsonifiable` dictionary.

Primitive and library instances can choose representation-specific encodings. For example, `nat` is encoded as a JSON number rather than as the unary constructor structure of Rocq natural numbers.

C.4 Generated Artifacts

For a declaration `T`, a successful command `Elpi derive.jsonifiable T` adds definitions for the encoder, decoder, round-trip proof, and registered instance. The visible instance name follows the generated-name convention used in the examples, such as `UserRole_Jsonifiable` or `BinTree_Jsonifiable`. Users should normally rely on typeclass resolution and the public operations shown in the `Jsonifiable` class rather than depending on internal helper names.

C.5 Interpreting Failure Categories

The benchmark artifact records non-success diagnostics with categories intended for triage. Some categories are semantic non-targets for a generic JSON serializer: propositions are erased at extraction, arbitrary functions do not have a canonical finite representation, sorts are type-level objects, and coinductive values may be infinite. Other categories are implementation limitations: dependent fields, indexed families, proof-bearing computational wrappers, arbitrary nested recursion, and mutual recursion require additional contracts or proof-generation techniques.